

Trabalho 1 – SD: Aplicação de Chat com gRPC e Protocol Buffers

Gabriel Souza Baggio¹, João Vitor Da Silva¹

¹Centro de Tecnologia – Universidade Federal de Santa Maria (UFSM)
Santa Maria – RS – Brasil

1. Visão Geral

Este documento descreve a implementação de uma aplicação de chat distribuída utilizando Java, gRPC e Protocol Buffers (protobuf). A aplicação segue o modelo cliente-servidor com suporte a múltiplos usuários em uma única sala de chat.

A comunicação entre clientes e servidor é realizada exclusivamente via gRPC, explorando tanto o modelo unário (`Register`, `SendMessage`) quanto o modelo de *streaming* do servidor (`ReceiveMessages`). O contrato de comunicação é definido no arquivo `contrato-chat.proto` e não pode ser alterado.

2. Arquitetura da Aplicação

A solução é composta por quatro componentes principais, conforme a Tabela 1.

Table 1. Componentes da aplicação

Componente	Arquivo	Responsabilidade
Servidor	<code>ServerChat.java</code>	Inicia o servidor gRPC na porta 50051 e registra o serviço
Serviço	<code>ChatServiceImpl.java</code>	Lógica de negócio: registro, envio e recebimento
Cliente	<code>ClientChat.java</code>	Interface do usuário: envio de mensagens e exibição em tempo real
Contrato	<code>contrato-chat.proto</code>	Define tipos de dados e RPCs

2.1. Contrato Proto

O arquivo `contrato-chat.proto` define três RPCs (*Remote Procedure Calls*) e quatro tipos de mensagem:

- `Register(User) returns (RegisterResponse)` – chamada unária para registro;
- `SendMessage(ChatMessage) returns (Ack)` – chamada unária para envio;
- `ReceiveMessages(User) returns (stream ChatMessage)` – *server streaming* para recepção contínua.

As mensagens `ChatMessage` contêm os campos `from` (remetente), `content` (conteúdo) e `timestamp` (`google.protobuf.Timestamp`), atendendo ao RFA04.

3. Fluxo de Funcionamento

3.1. Registro de Usuário (RFA01)

Ao iniciar, o cliente solicita um *username* via entrada padrão e realiza uma chamada unária `Register(User)`. O servidor verifica se o nome já está em uso num conjunto *thread-safe* (`ConcurrentHashMap.newKeySet()`). Se o nome for único, o usuário é registrado e a resposta indica sucesso; caso contrário, `success = false` e o cliente encerra a conexão.

Listing 1. Lógica de registro – ChatServiceImpl.java

```
private final Set<String> registeredUsers = ConcurrentHashMap.  
    newKeySet();  
  
if (registeredUsers.contains(username)) {  
    resposta = RegisterResponse.newBuilder()  
        .setSuccess(false).setUsername(username).build();  
} else {  
    registeredUsers.add(username);  
    resposta = RegisterResponse.newBuilder()  
        .setSuccess(true).setUsername(username).build();  
}  
responseObserver.onNext(resposta);  
responseObserver.onCompleted();
```

3.2. Conexão e Recebimento de Mensagens (RFA05)

Após o registro, o cliente inicia uma *thread* dedicada que chama `ReceiveMessages(User)`, abrindo um *stream* persistente. O servidor armazena o `StreamObserver` do cliente no mapa `observers`, indexado pelo *username*. Todo `ChatMessage` recebido pelo servidor é reencaminhado a todos os *observers* ativos.

Listing 2. Thread de recebimento – ClientChat.java

```
new Thread(() -> {  
    Iterator<ChatMessage> iterator = stub.receiveMessages(user);  
    while (iterator.hasNext()) {  
        ChatMessage msg = iterator.next();  
        System.out.println "[" + msg.getFrom() + "]: "  
            + msg.getContent();  
    }  
}).start();
```

3.3. Histórico de Mensagens

Ao se conectar, o novo cliente recebe automaticamente as últimas mensagens armazenadas. O servidor mantém uma fila circular de capacidade 50 (`LinkedBlockingQueue<ChatMessage>(50)`); mensagens mais antigas são descartadas quando a fila está cheia.

Listing 3. Envio do histórico ao novo cliente

```
for (ChatMessage msg : messageQueue) {  
    responseObserver.onNext(msg);  
}  
observers.put(username, responseObserver);
```

3.4. Envio de Mensagens (RFA03, RFA04)

O envio é feito via `SendMessage(ChatMessage)`. O servidor itera sobre todos os *observers* ativos e chama `onNext(request)` em cada um, garantindo que todos os usuários recebam a mensagem. O `Ack` de confirmação é retornado ao remetente, e a mensagem é adicionada ao histórico.

Listing 4. Broadcast da mensagem a todos os clientes — ChatServiceImpl.java

```
observers.values().forEach(observer -> {
    if (observer != null) observer.onNext(request);
});
Ack ack = Ack.newBuilder().setSuccess(true).build();
responseObserver.onNext(ack);
responseObserver.onCompleted();
if (messageQueue.remainingCapacity() == 0) messageQueue.poll();
messageQueue.offer(request);
```

3.5. Eventos de Notificação do Sistema (RFA07)

O servidor emite mensagens de sistema (`from = "System"`) em dois momentos:

1. **Entrada:** após o cliente ser adicionado ao mapa de *observers*, uma mensagem "`<username> has joined the chat!`" é enviada a todos.
2. **Saída:** o método `setOnCancelHandler()` detecta a desconexão do cliente, remove-o dos mapas e notifica os demais com "`<username> has left the chat!`".

Listing 5. Handler de desconexão — ChatServiceImpl.java

```
serverObserver.setOnCancelHandler(() -> {
    observers.remove(username);
    registeredUsers.remove(username);
    ChatMessage leave = ChatMessage.newBuilder()
        .setFrom("System")
        .setContent(username + " has left the chat!")
        .setTimestamp(ts2).build();
    observers.values().forEach(o -> { if (o != null) o.onNext(leave) });
});
```

4. Ordenamento de Eventos

4.1. Ordem por Remetente (RFA06)

A especificação exige que as mensagens preservem a ordem de envio por remetente. Dois mecanismos garantem isso:

- **gRPC serializa `onNext()` dentro de um *stream*:** chamadas `onNext()` num mesmo `StreamObserver` são serializadas e entregues em ordem, garantia do protocolo HTTP/2 subjacente.
- **`SendMessage` é síncrono:** o cliente usa o *stub* bloqueante `ChatServiceBlockingStub`, que aguarda o `Ack` antes de retornar. Um mesmo cliente não pode enviar mensagens fora de ordem ao servidor.

4.2. Ordem Global

A ordem global entre mensagens de remetentes distintos não é garantida formalmente. O servidor processa cada `SendMessage` na ordem de chegada ao processo gRPC e repassa imediatamente a todos os *observers*. Em cenários de alta concorrência, dois clientes enviando simultaneamente podem ser observados em ordens diferentes por cada receptor, conforme o escalonamento de *threads* da JVM.

4.3. Timestamp

O campo `timestamp` de cada `ChatMessage` é preenchido pelo cliente antes do envio, usando `System.currentTimeMillis() / 1000`. O servidor não o utiliza para reordenação; ele serve apenas para exibição.

5. Estruturas de Dados do Servidor

Table 2. Estruturas mantidas por `ChatServiceImpl`

Campo	Tipo	Finalidade
<code>observers</code>	<code>ConcurrentHashMap<String, StreamObserver<ChatMessage>></code>	Mapeia <i>username</i> ao <i>stream</i> ativo de cada cliente
<code>registeredUsers</code>	<code>Set<String></code> (via <code>ConcurrentHashMap</code>)	Controla nomes registrados para evitar duplicatas
<code>messageQueue</code>	<code>LinkedBlockingQueue<ChatMessage></code> (50)	Histórico circular das últimas 50 mensagens

O uso de estruturas *thread-safe* é essencial, pois múltiplos clientes realizam chamadas gRPC concorrentes. O `ConcurrentHashMap` garante operações atômicas sem bloqueios explícitos; o `LinkedBlockingQueue` oferece inserção e remoção *thread-safe* com controle de capacidade.

6. Limitações e Observações

- O histórico é mantido apenas em memória; reiniciar o servidor apaga todas as mensagens.
- Não há autenticação além do nome único.
- Capacidade da fila fixada em 50 mensagens; mensagens mais antigas são descartadas silenciosamente.
- Comunicação sem TLS (*Transport Layer Security*) (`usePlaintext()`): adequada para ambiente de laboratório.
- A ordem global entre remetentes distintos não é garantida em alta concorrência.

7. Atendimento aos Requisitos Funcionais

Abaixo segue uma tabela com um resumo dos requisitos funcionais do trabalho implementados.

Table 3. Mapeamento dos requisitos à implementação

ID	Requisito	Como é atendido
RFA01	Registro com nome único	<code>registeredUsers</code> rejeita duplicatas; resposta indica sucesso ou erro
RFA02	Sala única com múltiplos usuários	Único <code>ChatServiceImpl</code> ; <code>observers</code> guarda todos os clientes ativos
RFA03	Mensagens a todos via <code>SendMessage()</code>	Iteração sobre <code>observers.values()</code> com <code>onNext()</code>
RFA04	<code>ChatMessage</code> com <code>from</code> , <code>content</code> , <code>timestamp</code>	Campos preenchidos pelo cliente antes do envio
RFA05	<i>Stream</i> ativo via <code>ReceiveMessages()</code>	<i>Server streaming</i> mantém conexão aberta; <i>observer</i> salvo no mapa
RFA06	Ordem por remetente	<code>gRPC</code> serializa <code>onNext()</code> ; cliente aguarda <code>Ack</code>
RFA07	Notificações de entrada e saída	Mensagens de sistema via <i>join</i> e <code>OnCancelHandler</code>

8. Comandos para Compilar e Executar

8.1. Compilar

Na raiz do projeto (onde está o `pom.xml`):

Listing 6. Compilação do projeto

```
mvn clean package -DskipTests
```

O plugin `protobuf-maven-plugin` compila o `.proto` e gera as classes Java em `target/generated-sources`.

8.2. Servidor

Listing 7. Inicialização do servidor

```
# Via Maven
mvn exec:java -Dexec.mainClass="elc1018.grpc.chat.ServerChat"

# Ou via JAR
java -cp target/chat-1.0-SNAPSHOT.jar elc1018.grpc.chat.ServerChat
```

O servidor aguarda conexões na porta **50051**.

8.3. Cliente

Abra um terminal separado para cada cliente:

Listing 8. Inicialização do cliente

```
# Localhost (padrao)
mvn exec:java -Dexec.mainClass="elc1018.grpc.chat.ClientChat"
```

```
# Host remoto
mvn exec:java -Dexec.mainClass="elc1018.grpc.chat.ClientChat" \
    -Dexec.args="192.168.1.100"

# Via JAR
java -cp target/chat-1.0-SNAPSHOT.jar elc1018.grpc.chat.ClientChat
java -cp target/chat-1.0-SNAPSHOT.jar elc1018.grpc.chat.ClientChat
    192.168.1.100
```

Após iniciar, o cliente solicitará um *username*. Mensagens de outros usuários aparecem em tempo real. Para encerrar:

```
/exit
```